

Date: 07-09-2026

**Lab # 03 TO UNDERSTAND AND OBSERVE THE RUNNING TIME OF THREE
SIMPLE SORTING ALGORITHMS, SELECTION SORT, BUBBLE SORT AND INSERTION
SORT**

Objective:

- Understand and apply basic sorting algorithms.

Theory:

Sorting algorithms are methods used to arrange a list of items (like numbers) in a specific order, such as from smallest to largest. In this lab, we will explore three simple sorting methods: **Selection Sort**, **Bubble Sort**, and **Insertion Sort**.

1. **Selection Sort:** 1. This algorithm works by repeatedly finding the smallest (or largest) element from the unsorted portion of the list and swapping it with the first unsorted element.
2. Imagine you have a deck of cards, and you want to sort them by value. You look through the cards to find the smallest card and move it to the front, then repeat this process with the rest of the cards.
3. This takes some time because for every element, you have to compare it with many other elements to find the smallest one.

2. Bubble Sort:

1. Bubble Sort works by repeatedly going through the list and comparing each pair of adjacent elements. If the pair is in the wrong order (the first is bigger than the second), it swaps them.
2. Picture a row of bubbles of different sizes. The bigger bubbles "bubble up" to the top as they swap places with the smaller bubbles. The process continues until all the bubbles (or numbers) are in the right order.
3. This can take a while, especially if the list is long, because it has to go through the entire list multiple times.

3. **Insertion Sort:** 1. Insertion Sort builds the sorted list one item at a time. It looks at each element and inserts it into its correct position in the already sorted portion of the list.
2. Think about sorting a hand of playing cards. As you pick up each card, you place it in the correct position relative to the cards you're already holding.

3. If the list is almost sorted already, this algorithm can be pretty quick. But if it's all jumbled, it takes more time.

Procedure:

1. **Implementation of Sorting Algorithms:** Start by writing simple Python programs for each sorting algorithm.

a) Selection Sort:

```
def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

b) Bubble Sort:

```
def bubble_sort(arr):
    for i in range(len(arr)):
        for j in range(0, len(arr) - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

c) Insertion Sort:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
```

2. **Testing the Sorting Algorithms:** Test each sorting algorithm using a small list of numbers (e.g., 10-20 numbers).

Example list:

python

```
arr = [64, 25, 12, 22, 11]
```

3. **Measuring the Time:** Measure the time it takes to run each algorithm using Python's time module.

```
python
```

```
import time
```

```
start_time = time.time()  
selection_sort(arr) end_time  
= time.time()
```

```
print(f"Selection Sort Time: {end_time - start_time} seconds")
```

Exercise

1. Write a program to implement **Bubble Sort**. The program should take an array of integers from the user and sort the elements in **ascending order**. Display the original and sorted arrays.
2. Write a program to implement **Insertion Sort**. The program should take an array of integers as input and sort the elements in **ascending order**. Display the original and sorted arrays.
3. Write a program to implement **Selection Sort**. The program should take an array of integers from the user and sort the elements in **ascending order**. Display the original and sorted arrays.
4. Write a program to implement **Merge Sort** using the divide-and-conquer approach. The program should take an array of integers as input and sort the elements in **ascending order**. Display the original and sorted arrays.
5. What is the main difference between **Bubble Sort and Insertion Sort**?
6. Why is **Merge Sort** considered more efficient than Bubble Sort, Insertion Sort, and Selection Sort for large datasets?
7. How does **Selection Sort** select the element that should be placed at the correct position during each pass?
8. Which of the four sorting algorithms would you choose for:
 - a) A small and nearly sorted array
 - b) A large datasetGive a brief reason

Abstract:

This lab focuses on the implementation of different sorting algorithms in Python. The purpose of the lab was to understand and implement Bubble Sort, Insertion Sort, Selection Sort, and Merge Sort. Each program takes an array of integers from the user and sorts the elements in ascending order. Through these exercises, we learned how different sorting techniques work and compared their efficiency for different sizes of datasets. The programs were written and executed using Visual Studio Code (VS Code).

Procedure

Opened Visual Studio Code (VS Code) and created a new Python file.

1. Took an array of integers as input from the user.
2. Implemented Bubble Sort to arrange the elements in ascending order.
3. Implemented Insertion Sort to arrange the elements in ascending order.
4. Implemented Selection Sort to find and place the smallest element at the correct position.
5. Implemented Merge Sort using the divide-and-conquer approach.
6. Displayed the original array before sorting.
7. Displayed the sorted array after applying each sorting algorithm.
8. Executed the programs in VS Code and observed the output.
9. Checked and corrected any errors and verified that the results were correct.

Tasks:

Task 1

```
1
2 # Bubble Sort
3
4 arr = list(map(int, input("Enter integers separated by spaces: ").split()))
5
6 print("Original array:", arr)
7
8 n = len(arr)
9
10 for i in range(n):
11     for j in range(0, n - i - 1):
12         if arr[j] > arr[j + 1]:
13             arr[j], arr[j + 1] = arr[j + 1], arr[j]
14
15 print("Sorted array:", arr)
16
```

Problems Terminal Output Debug Console Ports

PS C:\Users\BSCS\Desktop\ai\c1> python -u "c:\Users\BSCS\Desktop\ai\c1\task.py"

Enter integers separated by spaces: 54 44 33 64

Original array: [54, 44, 33, 64]

Sorted array: [33, 44, 54, 64]

PS C:\Users\BSCS\Desktop\ai\c1> █

Task 2:

```
1 # Insertion Sort
2
3 arr = list(map(int, input("Enter integers separated by spaces: ").split()))
4
5 print("Original array:", arr)
6
7 for i in range(1, len(arr)):
8     key = arr[i]
9     j = i - 1
10
11     while j >= 0 and arr[j] > key:
12         arr[j + 1] = arr[j]
13         j -= 1
14
15     arr[j + 1] = key
16
```

Problems Terminal Output Debug Console Ports

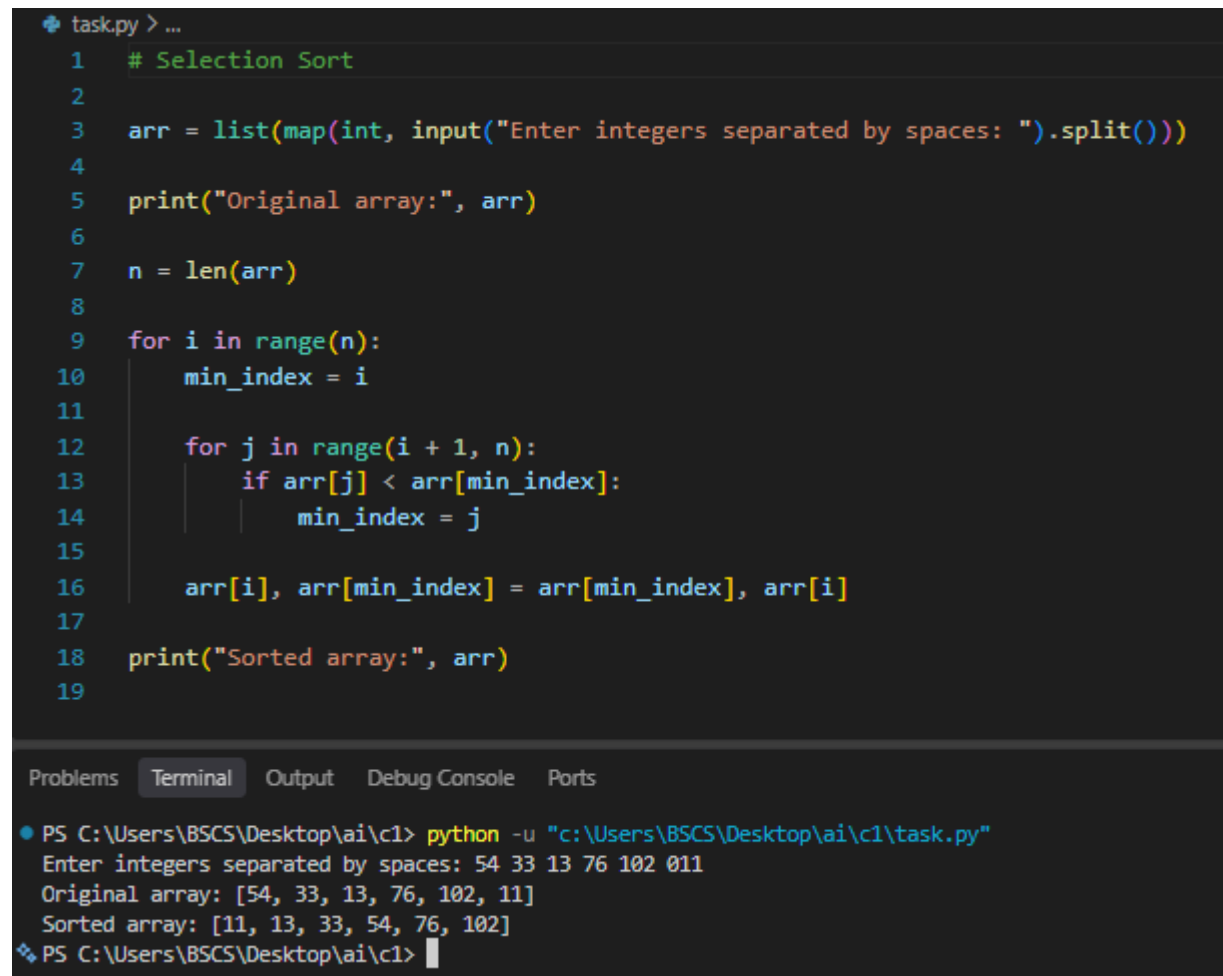
PS C:\Users\BSCS\Desktop\ai\c1> python -u "c:\Users\BSCS\Desktop\ai\c1\task.py"

Enter integers separated by spaces: 55

Original array: [55]

Sorted array: [55]

PS C:\Users\BSCS\Desktop\ai\c1> █

Task 3:

```
task.py > ...
1  # Selection Sort
2
3  arr = list(map(int, input("Enter integers separated by spaces: ").split()))
4
5  print("Original array:", arr)
6
7  n = len(arr)
8
9  for i in range(n):
10     min_index = i
11
12     for j in range(i + 1, n):
13         if arr[j] < arr[min_index]:
14             min_index = j
15
16     arr[i], arr[min_index] = arr[min_index], arr[i]
17
18 print("Sorted array:", arr)
19
```

Problems Terminal Output Debug Console Ports

● PS C:\Users\BSCS\Desktop\ai\c1> python -u "c:\Users\BSCS\Desktop\ai\c1\task.py"
Enter integers separated by spaces: 54 33 13 76 102 011
Original array: [54, 33, 13, 76, 102, 11]
Sorted array: [11, 13, 33, 54, 76, 102]
❖ PS C:\Users\BSCS\Desktop\ai\c1>

Task 4:

```
task.py > ...
1  def merge_sort(arr):
2      if len(arr) <= 1:
3          return arr
4
5      mid = len(arr) // 2
6
7      left = merge_sort(arr[:mid])
8      right = merge_sort(arr[mid:])
9
10     result = []
11     i = 0
12     j = 0
13
14     while i < len(left) and j < len(right):
15         if left[i] < right[j]:
16             result.append(left[i])
17             i += 1
18         else:
19             result.append(right[j])
20             j += 1
21
22     result.extend(left[i:])
23     result.extend(right[j:])
24
25     return result
26
27
28 arr = list(map(int, input("Enter integers separated by spaces: ").split()))
29
30 print("Original array:", arr)
31
32 arr = merge_sort(arr)
33
34 print("Sorted array:", arr)
```

Problems Terminal Output Debug Console Ports

```
PS C:\Users\BSCS\Desktop\ai\c1> python -u "c:\Users\BSCS\Desktop\ai\c1\task.py"
Enter integers separated by spaces: 55 66 77 88 99
Original array: [55, 66, 77, 88, 99]
Sorted array: [55, 66, 77, 88, 99]
PS C:\Users\BSCS\Desktop\ai\c1> |
```

5. Main difference between Bubble Sort and Insertion Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

Insertion Sort takes one element at a time and inserts it into its correct position among the already sorted elements.

6. Why is Merge Sort more efficient for large datasets?

Merge Sort uses the **divide-and-conquer** approach and has a time complexity of $O(n \log n)$.

Bubble Sort, Insertion Sort, and Selection Sort generally take $O(n^2)$ time for large or average datasets. Therefore, Merge Sort is usually much faster for large datasets.

7. How does Selection Sort select the correct element?

During each pass, Selection Sort searches the **unsorted portion of the array** for the smallest element.

It then swaps that smallest element with the first element of the unsorted portion.

8. Which algorithm would you choose?

a) Small and nearly sorted array:

Insertion Sort because it performs very well when the array is already nearly sorted.

b) Large dataset:

Merge Sort because its $O(n \log n)$ time complexity makes it more efficient for large datasets.

Conclusion:

In this lab, we successfully implemented Bubble Sort, Insertion Sort, Selection Sort, and Merge Sort in Python. The exercises helped us understand how different sorting algorithms arrange elements in ascending order. We also learned that Merge Sort is more efficient for large datasets, while Insertion Sort is useful for small and nearly sorted arrays. Overall, the lab provided practical experience in implementing and understanding basic sorting algorithms.